

## Our Approach

We chose to implement our agent using a minimax algorithm with alpha-beta pruning. There were several reasons for this choice. The main reason was that we were confident in our heuristic from Part A and thought that it would be a good challenge to expand and adapt it to minimax. Additionally, the deterministic nature of the minimax algorithm made it easier to test different versions of our agent against each other and understand why certain versions won over others.

We made five major design decisions in our minimax implementation: how to handle the different stages of the game differently, how to add nodes to the minimax tree, how wide the branching factor should be, what the cut-off depth should be, and what our evaluation function should be.

Our strategy for handling the different stages of the game was to hard code an opening 8 moves, search deep, but narrow in the middle game, and search wide but shallow in the late game. We chose to hard code our opening for several reasons. The main reason is that our evaluation function and the way we added nodes to the tree favored aggressive jumping, and we wanted to put our agent in a position to make those jumps. This is because jumping frequently increases synergy between frogs and reduces the total cost of reaching the end. Coding the openings was also an opportunity to get “free” moves, saving time for the agent to deliberate during the mid game.

After these opening moves ended, the mid game started, and our minimax function kicked in. We added nodes to the minimax tree by generating a list of boards for every possible move that could be made and then sorting those boards by our evaluation function. This precomputation and ordering made alpha-beta pruning more effective because the more promising nodes were explored first.

Our strategy in the mid-game was to search narrow, but deep. This strategy hinged on the argument that the mid game is more important than the late game, and therefore deserves more resources, and that making a better decision between fewer options is better than making a worse decision between many options. The evidence for the first part of the argument is that in the mid game, unlike the late game, the actions of an agent influence the actions of their opponent, and that because of this, decisions have higher stakes in the mid game. Our belief in the second part of the argument was based on our confidence in our evaluation function to determine the most promising moves on a given turn. We felt that searching only three moves during the mid game was acceptable if we knew those moves were among the best.

Our strategy in the late game was to search wide, but narrow. In the late game, which we defined as when all of an agent’s frogs are in the last three rows, it is less important to search deeply, because there is no need to consider what the opponent is doing. Conversely, it is more important to search wider because the value of moves flattens as an agent approaches victory and all its frogs are close to the end.

Many of our design decisions were made because of our confidence in our evaluation function, so it is worth exploring how it works. For a given board state, it estimates the net cost for the opponent to win compared to ourselves by subtracting the total estimated cost of all of our frogs from theirs. This is ideal for consideration by the minimax algorithm as our agent will assume the role of the “maximising” player and the opponent the “minimising” player. Our evaluation function from Project Part A (updated to also support blue frogs) is used to calculate the estimated cost of one frog. It is based on the A\* search algorithm, with a heuristic defined by this equation: Estimated cost for a space = distance from it to the end - two times the number of frogs with a lily pad behind them.

## Performance Evaluation

There were three main structural difficulties when it came to evaluating our agent. One tricky obstacle was the likelihood that red and blue have different optimal strategies. Another was the likelihood that different design decisions are not independent, for example one opening might work well with one mid-game strategy but poorly with another. The last obstacle was that there were a lot of these design decisions. To give some idea of the overall challenge, I will list here every single variable available for us to tune. Each of these variables should be considered as two variables, because the agent playing as red must be considered independently of the agent playing as blue. The number of reasonable options for each variable is listed before the variable.

- (4 options) Which of four openings to play
- (3 options) The growth factor in the progressive widening function in mid-game (will be explained in more detail in the next section)
- (3 options) The starting branching factor in mid-game
- (3 options) The cut-off depth in mid-game
- (2 options) When the late game should start
- (2 options) The cut-off depth in the late game
- (2 options) The branching factor in the late game

The combinations of these decisions give 864 unique agents for each color. To test which agent is best for blue and which is best for red would then require  $864^2 = 746,496$  tests from which we could tally up the record for each version of each agent.

Obviously, this was impossible for our assignment, so we needed to make some simplifying assumptions. The first was that the late game is independent from the early and mid-game and can therefore be tested independently. We can make this assumption because we only start the late game when all of the frogs have crossed a certain threshold, meaning that the late game looks similar to our agent regardless of the early and mid-game. We established that the maximum branching factor and a cut-off of 0 (basically just using A\* plus a growth function) was optimal.

The second simplifying assumption was that the mid-game variables can be linked into one “strategy” variable – wide and shallow, medium, and deep and narrow. We can make this assumption because of our time and space constraints. For each of these three strategies, there is a set of parameters that maxes out the resources available to us. It is reasonable to expect that the strategies that max out our resources will be the best strategies. For example, it would be unnecessary to test an agent with a branching factor of 3 and a cut-off depth of 6 against an agent with a branching factor of 6 and a cut-off depth of 6.

The last, and most difficult assumption, is that of the four openings we designed, the “grid” opening is the best for the agent when playing red and the “pedestal” opening is the best for the agent when playing blue.



*Red “grid” opening vs. Blue “pedestal” opening*

We made this assumption based on the theory that starting in a grid (or grid variation in the “pedestal” case) maximizes an agent’s jumping opportunities while minimizing its opponents. We tested this theory by assuming that the opening was independent and testing each of the four openings against each other. While imperfect, we believe this was the best strategy because the openings are more distinct from each other than the mid-game strategies are, and we thought we could still glean useful information about the general theory that grids are preferable to spreading frogs out in the early game.

Our tests showed that the “grid” opening was the best for the red agent (won three out of four matchups against its blue counterparts), and the “pedestal” opening was the best for the blue agent (won all four of its matchups).

With these simplifying assumptions made, this was the state of the variables needing testing:

- (3 options) mid game strategy
  - o Wide and shallow, middle, deep and narrow
- (2 options) when to start the late game
  - o All of an agent’s frogs in the last three rows or last four rows

This gave a more manageable  $6^2 = 36$  tests. When we conducted these tests, the winning combinations were:

Red: Deep and Narrow search with the late game beginning when all red frogs are in rows 5-7

Blue: Deep and Narrow search with the late game beginning when all blue frogs are in rows 0-2

### **Creative Additions from Outside Class Material**

An important creative aspect of our work was possible move ordering and filtering. After generating all possible moves, the Grow action and their corresponding board states, we had a very significant number of actions to select from. Feeding all of these into the minimax algorithm directly would have resulted in a prohibitively large branching factor and caused the algorithm to spend significantly more time to decide the next move, almost certainly exceeding the time constraint. Even after implementing optimizations such as alpha-beta pruning, branching factor and search depth cutoffs, a suboptimal decision may be reached as only a subset of possible actions could be considered, with priority determined by the order in which they were generated. To address this problem, we implemented possible move ordering and filtering prior to feeding them into the algorithm. All possible moves were analyzed by our evaluation function and the top moves with least estimated cost (3 in early game, 5 in mid-game and all in late game) and the Grow action are passed into the minimax algorithm. This ensures that the limited time resource is used to consider actions that are most likely to be the most optimal.

Another creative addition was the progressive widening of the branching factor as we searched deeper. This is an idea borrowed from implementations of Monte Carlo Tree Search (Coulom 2007; Couetoux et al. 2011a). In Monte Carlo Tree search implementations that use progressive widening, or, as it is often called, progressive unpruning, search nodes on more promising routes more widely. The same logic applies to our minimax implementation where, in the mid-game, only three children nodes are initially added to the root node. As we search deeper, these children nodes add more than three children nodes to themselves until we get to the end of the search where the nodes each consider five possible options. The main difference in implementation between our minimax code and a traditional Monte Carlo Tree Search algorithm with progressive widening is how the respective programs choose which nodes to progressively widen from. In MCTS, this is based on how many times the node is visited in the selection/expansion phase, in our code, it is based on what our evaluation function believes are the most promising nodes. The function itself is fairly simple, getting the branching factor by adding a starting branching factor to a growth factor times the current depth. We chose this strategy because its simplicity allowed us to test different versions of it easily. The three mid-game strategies discussed above had starting branching factors of 3, 4, and 6 and growth factors of  $1/4$ ,  $1/3$ , and  $1/2$  for the three strategies of narrow but deep, medium, and wide but shallow strategies respectively.

The last creative addition that we want to mention involves a tuning born from observing the agent through testing at the various stages of the engineering process. This was less academic and more based on “feel”, but it still constitutes a meaningful change that improved the performance of our agent. This addition stipulated that the agent must move all of its frogs into the last 3 rows (which was when we decided the late game would start) before it could move any

of the frogs in the last 3 rows. We added this stipulation after noticing that our agent was leaving frogs behind, and that this was hurting the performance of the agent. Our evaluation function left frogs behind because its inclination is to maximize aggressive jumping while minimizing the jumping of the opponent. This often led to a game where the agent would push the first 2-3 frogs up the board quickly and leave the other 2-3 behind. Once this happened, the agent would push those 2-3 frogs to the end, because moving the 2-3 that were left behind would expose jumping opportunities for the opponent. With more computing power, we would be able to search more options for more frogs more deeply, and this would likely fix the problem. With the current set of constraints, though, and our decision to hyperfocus on potential moves the evaluation function likes in the short term, this was not possible, and we needed to force the agent to choose amongst options it considered less promising.

## **Supporting Work**

In order for our agent to work correctly and play only legal moves, it needed an accurate internal representation of the current board state. Due to the variety of different actions that can be played, ensuring that its internal board state was correctly updated was a continuous endeavour. The agent is unable to access the board stored in the ‘referee’ program so we developed a ‘print\_board()’ function that prints the agent’s internal board state at the start of every turn when debugging to detect any discrepancies. Other key information about the state of the agent was also printed out during debugging as needed.

We have also saved the original working version of our agent with minimal optimisations, ‘agent\_base’, separately from our current version to compare their performance as we implemented improvements. This helped us determine if our optimisations materially improved the agent’s performance. For example, although some improvements, such as adding a late game heuristic to the evaluation function that also considers the cost of moving a frog laterally to reach an unoccupied cell in the final row, would theoretically yield more effective moves in the late game, empirical testing showed that any improvement to decision making was vastly overshadowed by the increased complexity and uncertainty of calculating this additional cost. These comparisons essentially ensured that optimisations that do not result in tangible performance improvements were not added to our final agent.

## **References**

*Coulom, R. 2007. Computing elo ratings of move patterns in the game of Go. In Computer Games Workshop.*